

D-2

Date: 2026.../09.../18.

Classes:

Syntax:

```
class classname {  
    fun funname() {  
        operation  
    }  
}
```

Object

```
fun main() {
```

```
class classname {  
    object / function / features  
}
```

~~82~~

~~this = the class~~

Syntax:

* private - var variable : datatype = data

get() - ~~var~~ custom getter can be used in
~~field()~~ same line or different line.

Field - Memory storage. Unlike with setter, not
any random variable can be used.

set() - Set value

~~value = value generated by Kotlin~~

```

class User {
    var name : String = ""
    val sname : String
    get() = "$name"
    val fname : String
    get() = "$name $sname"
}

```

Here, when executed, the value for sname will be set what is get() by name. And it will be its value for complete execution, as seen because of fname. when using get(), no variable should be empty and have no init data. $\emptyset$

For each time the function is called, var's despite being val being used, the data for the variable changes. while on the other hand when using normal data allocation, the variable's data will remain constant.

In page 111 ~~111~~, when the program is run, intended output isn't generated.

* Constructor: It is used to pass data called when ~~the~~ it is not passed when creating the class.

Syntax:

```

class classname {
    constructor (var: vartype) {}
}

```

for Eg:

```
class User {
```

```
    var name: String = ""
```

```
    var surname: String = ""
```

```
    constructor (name: String, surname: String) {
```

```
        this.name = name
```

```
        this.surname = surname
```

```
    }
```

```
}
```

```
fun main() {
```

```
    val user = User("J", "D")
```

```
    println(user.name)
```

```
    println(user.surname)
```

```
}
```

But we can also just

```
class User (name: String, surname: String)
```

this is because it is a special kind of constructor, primary constructor where the constructor is called after calling class and using constructor keyword is optional.

* Extension function: add some function without much change to the main function.

Syntax:

```
fun Datatype.funname(): Datatype = operations
```

* Scope function: allows to temporarily create a scope around an object and execute some code. We can do this by using:

~~this~~ it

1> let 2> apply 3> run 4> also 5> with.

You can access the object by using this and it.

* let: Syntax:

```
val variable = prevvariable ? . let &
                                operation
&
```

* apply: used to initialize objects at the time of creation rather than later in code.

~~for Eg~~ for Eg:

```
val client = Client, apply &
    token = "asdf"
    connect()
    authenticate()
&
```

&

```
fun main() {
```

```
    client.getdata()
    &
```

* ~~Def~~: * Secondary Constructor: another constructor other than the primary constructor. But it needs to call the primary constructor using this.

- Syntax:

```
constructor (var: vartype/class): this(class, var) {
    // body
    &
```

&

* Inner Classes:

We can create class inside class. These are static.

Syntax

```
class class1 {
    class class2 {
        fun funname () {
            // Body
        }
    }
}
```

```
fun main () {
    val var = class1().class2()
    var.funname()
}
```

If we want to use data passed to outer class in inner class we need to use the code name inner.

* Inheritance: many classes of objects share the same characteristics. The class that inherits from another class is known as subclass or child and the one it inherits from is known as superclass or parent.

In Kotlin all classes are closed. So we have to use open keyword to allow inheritance from.

for Eg:

```
open class Mammal {
    val haveHairOrFur = true
    val warmblooded = true
    var canFeed = false
}
```

```

fun feedYoung() {
  if (canFeed) {
    println("feeding young with milk")
  }
}

```

```

class Dog : Mammal() {
  fun makeVoice() {
    println("Bark Bark")
  }
}

```

```

fun main() {
  val dog = Dog()
  dog.makeVoice()
  println(dog, have Hair Or Fur)
  println(dog, warm Blooded)
  val mammal : Mammal = dog
  mammal.canFeed = true
  mammal.feedYoung()
}

```

Syntax:

```

class class 1 { }

```

```

class class 2 : class 1 { }

```

• to override elements, override function has to be given. But it ~~override fun~~ has to be opened using open.

- When inheriting from nonempty constructor, we need to define ~~Syntax~~ the arguments.
- to call the super function used in parent class, we use super keyword. This is only used if we have used override.
- Abstract classes are those that cannot be used in ~~an~~ and is only there to be superclass.
- Multiple inheritance not possible.
- Interface is a set of properties that has to be inherited. We use interface keyword for this.
- val can be overridden by var.
- When 2 interface has a method with same name and parameters, then when class implements this must first override. then call using super.

* Visibility: 1> Public: visible everywhere for clients which can see the declaring class.

2> Private: visible inside the class only.

3> Protected: visible inside this class and subclasses.

4> Internal: visible inside this module for clients which can see the declaring class.

• hashCode - gives hashCode

toString -

• Copy() - creates a temporary clone of data class that we can edit without change in

the original. For Eg:

```
val p = Player(0, "Gecko", 9999)
println(p.copy(points = points + 1)) // 0, Gecko, 10000
println(p) // 0, Gecko, 9999
```

* KotlinVersion.CURRENT // Version of Kotlin

* import java.util.Date

~~import~~

Date() // (Date using System "Day Month Date Time Year")

* Object: Objects are instances of class. We can create objects using object expression and object declaration.

- Object ^{Expression} declaration: We make an object without making a class, and giving no name to object.

Syntax:

```
val var = Object {}
```

- Object Declaration: we make an object like when using object expression but it is not anonymous and as such we can call it

Syntax:

Object Objname {

var vari = data

}

```
fun main() {
```

```
    print(Objname.vari)
```

}

- Companion Object is something attached to a class and not to the object. It lets you call functions like `User.empty()` without making an instance first. And unlike Java's static stuff, it can inherit, so you can share logic across classes.

* Exceptions in Kotlin: unwanted event that interrupts the flow of event.

- Arithmetic Exception = When the ~~error~~ exception is arithmetic.
- NumberFormatException = occurs when converting string into numbers but the string is not just numbers.
- We can also throw/display the exception using throw
Syntax: `throw ExceptionType (Message to display)`
- Throwing is used in condition that function is not prepared for.
- We can also define our own exceptions using throw.
- We can also catch exceptions using try and catch blocks -
Same as Java. Syntax:

try {

body

} catch (e: Exception) {

body

}

- It can also be used as an expression.
- We can also use finally in try-structure. It is to invoke something despite the exception.
- IllegalArgumentException = when an argument has an incorrect value.

- IllegalStateException = used when state of our system is incorrect.
- error function =
- Hierarchy of exceptions:
 - Error - exceptions that are impossible to recover from
 - Exception - exceptions we can recover from.

Enum Classes: It is used to represent a fixed state of value. we use UPPER_SNAKE_CASE (in uppercase and separated by -)

- has the following companion object functions:
 - values - returns an array of all values of enum class.
 - valueOf - passes a string into a value matching its name or throws an exception.
- Instead of these, we can also use:
 - enumValues -
 - enumClasses -